

1. Modulo di Rescheduling del Materiale Rotabile (Rolling Stock) Il primo passo della sequenza gestisce i treni e si basa su un approccio algoritmico *greedy* (goloso). Puoi prendere come riferimento l'algoritmo sviluppato in precedenza da Roberto. In questo modulo devi implementare:

- **Fissaggio di un Seed:** Dato che l'algoritmo di partenza è un "greedy randomizzato", devi impostare un seed (seme) iniziale. Questo è fondamentale nelle fasi di sviluppo per sapere esattamente cosa sta succedendo nel codice e avere risultati riproducibili.
- **Ciclo iterativo di assegnazione:** Devi avviare un loop continuo (un ciclo) in cui provi ripetutamente ad assegnare uno specifico viaggio (trip) a una specifica locomotiva.
- **Controlli di fattibilità (Feasibility checks):** Anche se il cuore della logica greedy occupa poche righe di codice (circa 20), la parte più complessa che devi implementare riguarda i rigorosi controlli di fattibilità per confermare o rifiutare le assegnazioni.
- **Gestione degli scarti:** Il loop continua fino a quando non è più possibile fare assegnazioni; a quel punto, il codice deve attuare un comportamento diverso per sbloccare la situazione, gestendo variazioni o modifiche al piano.
- L'approccio che deve essere utilizzato è contenuto in `RandomizedGreedy()` in `frish_solution.cc` che deve essere convertito in linguaggio Python perché sarà il linguaggio dell'approccio iterativo

2. Passaggio dei dati e Modulo di Rescheduling dell'Equipaggio (Crew) Una volta completato il piano per il materiale rotabile, l'output generato (che conterrà delle differenze rispetto alla situazione normale, come cancellazioni o aggiunte di viaggi) deve essere passato al secondo modulo della sequenza. Per questa fase devi implementare:

- **Un secondo approccio Greedy:** Per il crew scheduling devi strutturare un processo molto simile a quello del primo modulo, tentando di assegnare i task all'equipaggio. Come scheletro di partenza, puoi utilizzare il codice greedy iniziale che Manuel aveva scritto all'interno della sua funzione `VNS. (calculateInitialSolution())`
- **Logica di scarto immediata:** Essendo un algoritmo greedy basilare, non devi complicare la logica: se un task non può essere assegnato a un membro dell'equipaggio, l'algoritmo deve semplicemente cancellarlo.
- Per la c

3. Valutazione e uniformità tecnica Una volta che entrambi i moduli hanno concluso le loro assegnazioni a cascata, il sistema deve chiudere il processo e **valutare il punteggio (score) finale** della simulazione.

Dal punto di vista tecnico, c'è un vincolo fondamentale che devi rispettare: **tutto l'ecosistema sequenziale deve essere implementato nello stesso linguaggio di programmazione**. Dal momento che l'algoritmo di Roberto è scritto in C++ (che garantisce prestazioni molto superiori rispetto a Python o Java), il team concorda sulla necessità di uniformare il codice per poter eseguire una comparazione valida e corretta delle prestazioni degli algoritmi